JOB PORTAL PROJECT REPORT

Using React JS, Java JSP/Servlet, MVC Architecture, and MySQL

Academic Report Document

Project Title: Job Portal
Category: Beginner-Level Full Stack Web Project
Frontend: React JS
Backend: Java JSP + Servlet + MySQL

This report explains the project summary, system design, file structure, MVC model, and workflow.

Contents

1. Introduction
2. Project Overview
3. Problem Statement
4. Objectives
5. Technology Stack
6. Key Features
7. Current Working Model
8. Project Modules
9. File Structure
10. Frontend Architecture
11. Backend MVC Structure
12. Database Design
13. Interconnection of Modules
14. System Flow
15. MVC Data Flow
16. Working of Important Functionalities
17. Advantages of the Project
18. Limitations
19. Future Enhancements
20. Testing and Verification
21. Conclusion
22. Quick Summary

Title

Job Portal Using React JS, Java JSP/Servlet, MVC, and MySQL

Abstract

This project is a beginner-friendly Job Portal developed as a simple full-stack academic application. The aim of the project is to convert a static job portal design into a dynamic system where users can view jobs, search for jobs, and apply for them, while an admin can add and delete job postings. The frontend is developed using React JS with functional components, React Router, and simple CSS. The backend architecture is designed using Java JSP and Servlets following the MVC pattern, with MySQL as the database.

For practical usability and smooth demonstration, the current running frontend version uses browser localStorage to manage users, jobs, and applications. This makes the portal fully functional without requiring backend deployment during the demo. At the same time, the repository also includes a complete beginner-level Java backend structure with Servlet controllers, DAO classes, utility classes, JSP view, and SQL schema, so the project still matches the original full-stack academic design and can be explained properly in viva.

This project demonstrates routing, login flow, protected pages, CRUD-style job handling, data storage concepts, MVC architecture, JDBC connectivity, and the interaction between frontend, backend, and database layers.

1. Introduction

A job portal is a platform where job seekers can view available job openings and submit their applications, while administrators can manage job postings. In real-world systems, job portals often include many advanced features such as resume upload, company profiles, notifications, and analytics. However, for learning purposes, this project keeps the system simple and easy to understand.

The main goal of this project is to demonstrate how a static website can be turned into a dynamic web application using modern frontend development and traditional Java web development concepts. The React frontend focuses on better user interaction and navigation, while the Java backend follows MVC architecture to explain structured full-stack development.

2. Project Overview

This project is divided into three main parts:

Frontend

Built with React JS. It provides pages for home, jobs, login, registration, applications, and admin dashboard.

Backend

Built with JSP and Servlets using MVC architecture. It includes model classes, controller servlets, DAO classes, utility classes, and minimal JSP support.

Database

Designed in MySQL with jobs and applications tables.

The current working version of the portal supports:

- user registration

- user login
- job search
- job application
- application tracking
- admin job management

The frontend currently works independently using browser localStorage, while the backend remains available in the project as the original MVC implementation reference.

3. Problem Statement

The original project was a static HTML/CSS job portal with a navbar, hero section, search area, job cards, and apply buttons. Since the jobs were static, the portal did not support real data handling, job search logic, applications, or administration.

The problem was to convert this static design into a simple, beginner-level full-stack project with:

- React frontend
- Java JSP/Servlet backend
- MVC architecture
- MySQL database

The project also needed to remain simple enough for first-year or beginner-level explanation during viva.

4. Objectives

The main objectives of the project are:

- To create a dynamic job portal from a static design
- To implement a clean React frontend using functional components
- To provide a simple login and registration system
- To allow users to search and apply for jobs
- To allow admin to add and delete jobs
- To maintain a beginner-friendly code structure
- To demonstrate MVC theory using Java JSP and Servlets
- To provide a project that is easy to explain in viva

5. Technology Stack

Frontend

- React JS
- React Router DOM
- JavaScript
- CSS

Backend

- Java
- JSP
- Servlet
- JDBC

Database

- MySQL

Tools

- Node.js and npm
- Create React App
- Eclipse IDE / VS Code
- Apache Tomcat

6. Key Features

User Side Features

- User registration
- User login
- View all jobs
- Search jobs by title
- Search jobs by location
- Apply for a job
- View own applications

Admin Side Features

- Admin login
- View admin dashboard
- Add new job
- Delete job
- View job application records

System Features

- Route-based navigation
- Role-based protected pages
- Reusable components
- Simple persistent storage in browser for demo
- Backend MVC structure for academic explanation

7. Current Working Model

The project currently runs in a hybrid academic format:

- The **live frontend** is fully working and stores data in browser `localStorage`
- The **backend folder** contains the JSP/Servlet/MySQL MVC structure designed for the original full-stack requirement

This approach is useful because:

- the project can be demonstrated immediately
- the user experience is complete
- the academic backend architecture is still present
- viva explanation remains aligned with the original task

8. Project Modules

8.1 Authentication Module

This module handles:

- registration
- login
- logout
- session-like user state in frontend
- role-based access

8.2 Job Management Module

This module handles:

- listing jobs
- searching jobs
- adding jobs
- deleting jobs

8.3 Application Module

This module handles:

- applying to jobs
- storing application data
- tracking submitted applications

8.4 Admin Module

This module handles:

- job control
- application review
- portal statistics display

9. File Structure

```
frontend/
├── backend/
│   ├── WebContent/
│   │   ├── index.jsp
│   │   └── WEB-INF/
│   │       └── web.xml
│   ├── src/com/jobportal/
│   │   ├── controller/
│   │   │   ├── AddJobServlet.java
│   │   │   ├── ApplyJobServlet.java
│   │   │   ├── DeleteJobServlet.java
│   │   │   └── GetJobsServlet.java
│   │   ├── dao/
│   │   │   ├── ApplicationDAO.java
│   │   │   └── JobDAO.java
│   │   ├── model/
│   │   │   └── Job.java
│   │   └── util/
│   │       ├── DBConnection.java
│   │       └── JsonUtil.java
│   └── README.md
├── database/
│   └── job_portal.sql
├── docs/
```

```
│   ├── 01-react-files.md
│   ├── 02-backend-files.md
│   ├── 03-misc-files.md
│   ├── 04-viva-preparation.md
│   └── PROJECT_REPORT.md
├── public/
│   └── index.html
├── src/
│   ├── components/
│   │   ├── JobCard.js
│   │   ├── JobList.js
│   │   ├── Navbar.js
│   │   ├── PortalSummary.js
│   │   └── ProtectedRoute.js
│   ├── context/
│   │   └── AuthContext.js
│   ├── pages/
│   │   ├── AdminPanel.js
│   │   ├── Applications.js
│   │   ├── Home.js
│   │   ├── Jobs.js
│   │   ├── Login.js
│   │   └── Register.js
│   ├── services/
│   │   └── portalStore.js
│   ├── App.js
│   ├── App.test.js
│   ├── index.css
│   ├── index.js
│   ├── reportWebVitals.js
│   └── setupTests.js
├── package.json
└── README.md
```

10. Frontend Architecture

The frontend is built using React JS and follows a modular structure.

Main frontend files and responsibilities

- `index.js`

Starts the React application

- `App.js`

Handles routing and wraps the app with authentication context

- `AuthContext.js`

Maintains login state and auth-related functions

- `Navbar.js`

Provides navigation based on user status

- `ProtectedRoute.js`

Restricts certain routes to logged-in users or admin

- `Home.js`

Shows landing page, recent jobs, and statistics

- `Jobs.js`

Displays job listings and search/apply features

- `Applications.js`

Displays jobs applied by the current user

- `AdminPanel.js`

Shows admin dashboard and job control features

- `portalStore.js`

Manages local data like jobs, users, applications, and current user

11. Backend MVC Structure

The backend is designed according to MVC architecture.

11.1 Model

File: backend/src/com/jobportal/model/Job.java

Purpose:

- Represents one job object
- Stores fields like id, title, company, location, and description

11.2 View

File: backend/WebContent/index.jsp

Purpose:

- Acts as a simple JSP view
- Shows backend status page

11.3 Controller

Files:

- `GetJobsServlet.java`
- `AddJobServlet.java`
- `DeleteJobServlet.java`
- `ApplyJobServlet.java`

Purpose:

- Receive frontend requests
- Process input
- Call DAO methods
- Return JSON response

11.4 DAO Layer

Files:

- `JobDAO.java`
- `ApplicationDAO.java`

Purpose:

- Perform SQL operations
- Fetch jobs
- Insert jobs
- Delete jobs
- Insert applications

11.5 Utility Layer

Files:

- `DBConnection.java`
- `JsonUtil.java`

Purpose:

- `DBConnection` creates MySQL connection using JDBC
- `JsonUtil` escapes strings for valid JSON output

12. Database Design

The project uses MySQL with two main tables.

12.1 Jobs Table

Column | Type | Description

id | INT | Primary key, auto increment
title | VARCHAR(100) | Job title
company | VARCHAR(100) | Company name
location | VARCHAR(100) | Job location
description | TEXT | Job details

12.2 Applications Table

Column | Type | Description

id | INT | Primary key, auto increment
job_id | INT | Foreign key linked to jobs table
applicant_name | VARCHAR(100) | Name of applicant

SQL Script

The SQL schema is stored in:

[database/job_portal.sql](/Users/pahul/frontend/database/job_portal.sql)

13. Interconnection of Modules

The current working frontend flow is:

public/index.html -> index.js -> App.js -> AuthContext + Navbar + routes -> pages -> portalStore.js -> browser localStorage

Authentication Flow

Login.js / Register.js -> AuthContext.js -> portalStore.js -> localStorage

Job Flow

Jobs.js -> JobList.js -> JobCard.js

Data comes from:

Jobs.js -> portalStore.js

Application Flow

Jobs.js -> apply form -> portalStore.js -> saved application -> Applications.js

Admin Flow

AdminPanel.js -> portalStore.js -> jobs/applications/statistics

14. System Flow

14.1 Current Demo System Flow

- 1. User opens the React app
- 2. React loads main route structure
- 3. Auth context checks if a user is already saved
- 4. User can browse jobs or login/register
- 5. After login:
- user can apply to jobs
- user can view own applications
- admin can manage jobs
- 6. All data is saved in browser localStorage

14.2 Intended Full Stack System Flow

- 1. User opens React frontend
- 2. React sends request to Servlet API
- 3. Servlet receives request
- 4. Servlet calls DAO method
- 5. DAO talks to MySQL using JDBC
- 6. MySQL returns result
- 7. Servlet sends JSON response
- 8. React updates UI

15. MVC Data Flow

The MVC flow in the backend is:

Controller

Servlet receives request from frontend

DAO / Model

DAO gets or updates database data using model objects

View / Response

Data is returned as JSON or shown through JSP

In simple words:

Frontend -> Controller -> DAO -> Database -> Controller -> Frontend

16. Working of Important Functionalities

16.1 Registration

- User enters name, email, password
- `Register.js` sends data to `AuthContext`
- `AuthContext` calls `registerUser` in `portalStore`
- New user is saved
- User is redirected to jobs page

16.2 Login

- User enters email and password
- `Login.js` calls `login` in `AuthContext`
- Auth context verifies user with `portalStore`
- User session is stored
- User is redirected based on role

16.3 Job Search

- User enters search text or location
- `Jobs.js` updates search parameters
- Jobs are filtered using `getJobs`
- Matching jobs are displayed

16.4 Apply for Job

- Logged-in user clicks Apply
- Application form opens
- Form data is stored through `applyForJob`
- Application is shown in My Applications page

16.5 Admin Adds Job

- Admin fills job form
- `AdminPanel.js` calls `addJob`
- New job is saved
- Jobs list refreshes

16.6 Admin Deletes Job

- Admin clicks delete
- `deleteJob` removes job
- Related applications are also removed from storage in current demo flow

17. Advantages of the Project

- Simple and beginner-friendly
- Easy to explain in viva
- Clear separation of frontend and backend layers
- Reusable component design
- Includes login and role-based navigation
- Includes both working demo logic and academic backend architecture

18. Limitations

- Current live demo uses browser storage instead of deployed backend database
- No password encryption
- No file upload or resume support
- No edit/update job feature
- No advanced validation
- No production-level security

19. Future Enhancements

- Connect React login and jobs directly to Java Servlet backend
- Add MySQL-based user table and real backend authentication
- Add update/edit job feature
- Add resume upload
- Add status tracking for applications
- Add search filters such as salary and job type
- Add password hashing and session management

20. Testing and Verification

The frontend has been verified using:

- `npm run build`
- `npm test`

This confirms that the React app compiles successfully and basic rendering works.

21. Conclusion

The Job Portal project successfully transforms a basic static design into a dynamic, structured academic web application. It demonstrates important concepts such as React component design, route management, authentication flow, role-based navigation, CRUD-like data handling, MVC architecture, Servlet-based controllers, DAO-based database operations, and SQL schema design.

The current version is especially useful for demonstration and viva because it works immediately in the browser while still preserving the original full-stack architecture expected from the assignment. This makes the project both practical and academically strong.

22. Quick Summary

Project Summary

A beginner-level Job Portal built using React JS for frontend and designed with Java JSP/Servlet MVC backend and MySQL database support.

Project Overview

The project supports job browsing, user authentication, job application, and admin job management. The running frontend uses browser storage for a complete demo, while the backend MVC code is included for full-stack explanation.

Key Features

- Login and registration
- Job search

- Job application
- My Applications page
- Admin dashboard
- Add and delete jobs
- Role-based protected pages

MVC Structure

- Model: `Job.java`
- View: `index.jsp`
- Controller: Servlet classes
- DAO: `JobDAO`, `ApplicationDAO`
- Utility: `DBConnection`, `JsonUtil`

System Flow

Frontend flow:

React UI -> AuthContext / portalStore -> localStorage

Backend flow:

React -> Servlet -> DAO -> JDBC -> MySQL -> JSON response